

# Counterfactual Residual Flow Steering for Collision-Aware Vision–Language–Action Policies

Draft Research Proposal and Falsification Plan

Anonymous Draft

Replace with author names and affiliations before submission

**Abstract**—Flow-matching vision–language–action (VLA) policies such as  $\pi_{0.5}$  generate continuous action chunks by transporting Gaussian noise to an action distribution. Existing safety mechanisms for VLA policies either filter completed actions with analytic constraints or solve constrained optimization problems during generation. A simpler learned risk probe or best-of- $N$  seed selector cannot create a safe detour when the base distribution assigns negligible probability to such behavior. We propose CRFS (Counterfactual Residual Flow Steering), a deliberately minimal research formulation for learning a differentiable safety direction inside a frozen flow-matching VLA. An offline oracle first constructs the nearest collision-free action prefix that preserves the nominal local endpoint. This constrained projection has one objective—minimum action change—and treats safety and endpoint preservation as hard constraints. The resulting action displacement supervises a small steering network with a single squared-error loss. At inference, the predicted displacement is injected as a residual velocity over the remaining flow integration, thereby altering the generated action distribution without an online control-barrier-function solver or quadratic program. The proposal is organized as a sequence of falsifiable gates: measurement validity, counterfactual feasibility, oracle steerability, learned steerability, and closed-loop utility. Each gate contains explicit stop or refinement criteria so that failure can be localized before scaling to additional tasks, obstacles, or full-arm safety.

## I. MOTIVATION

Vision–language–action models map visual observations and language instructions to robot actions, but collision avoidance is not automatically guaranteed by imitation learning or broad pretraining. VLASA/AEGIS addresses this problem by placing a control-barrier-function (CBF) safety-constraint layer after the nominal VLA output and solving a quadratic program (QP) that minimally modifies unsafe actions [1]. SafeLIBERO was introduced to evaluate this setting, with obstacles placed near targets or directly in the movement path [1]. The released evaluation code uses  $\pi_{0.5}$  as a base policy, requests an action chunk, and executes the first five actions before replanning [2].

The safety filter is effective, but it does not teach the generative policy a collision-avoidance behavior. A recent constrained-flow approach instead evaluates predicted trajectories during denoising and solves a minimum-norm constrained optimization problem inside the generation loop [3]. This provides predictive correction, but still requires an analytic online solver.

A learned scalar risk probe is not sufficient by itself. It may predict that every available sample is unsafe, but it does not specify a task-preserving direction in action space. Similarly,

best-of- $N$  sampling only chooses among behaviors already represented by the base distribution. If a useful detour has negligible probability under the frozen policy, sampling more seeds selects the least unsafe behavior rather than synthesizing a safe one.

This proposal asks a narrower and falsifiable question:

Can a small learned residual, supervised by minimal safe counterfactual actions, causally steer an intermediate  $\pi_{0.5}$  action-flow state toward a collision-free action while preserving the nominal local endpoint?

The proposed method is not intended to provide a formal hard-safety guarantee. Its purpose is to test whether collision-avoidance behavior can be represented as a learned, differentiable direction inside the action-generation flow.

a) *Proposed contributions.*: The draft investigates three claims:

- 1) A clean counterfactual supervision target can be defined without a weighted engineering reward: the nearest safe, endpoint-preserving action prefix.
- 2) That action displacement can be injected as a residual flow velocity and can causally rescue a fixed observation–noise pair.
- 3) A compact steering model can learn the displacement with one regression objective and improve closed-loop safe task success without materially degrading task completion.

## II. POSITIONING RELATIVE TO EXISTING SAFETY LAYERS

AEGIS identifies a task-relevant obstacle, reconstructs its geometry, and applies a CBF-QP after the VLA action output [1]. Constrained flow matching applies symbolic trajectory constraints and online minimum-norm correction during iterative action generation [3]. In contrast, CRFS uses constrained optimization only *offline* to create counterfactual labels. Deployment uses only a learned residual network and the frozen VLA sampler.

## III. PROBLEM SETUP

### A. Frozen flow-matching action policy

Let  $o$  denote the visual observation,  $\ell$  the language instruction, and  $s$  the robot proprioceptive state at a replanning instant. A flow-matching VLA defines a conditional velocity field [4]

$$\frac{dx_t}{dt} = v_\theta(x_t, t \mid o, \ell, s), \quad t : 1 \rightarrow 0, \quad (1)$$

TABLE I  
CONCEPTUAL POSITIONING. AN ONLINE SOLVER IS AN OPTIMIZATION  
PROBLEM SOLVED DURING DEPLOYMENT.

| Method               | Intervention          | Online solver | Guarantee        |
|----------------------|-----------------------|---------------|------------------|
| AEGIS [1]            | After VLA output      | Yes           | Conditional      |
| Constrained flow [3] | During denoising      | Yes           | Constraint-based |
| Best-of- $N$         | Sample selection      | No            | No               |
| CRFS                 | Learned residual flow | No            | No               |

where  $x_1 = \epsilon \sim \mathcal{N}(0, I)$  and  $x_0 = A$  is an action chunk. The  $\pi_{0.5}$  family combines a pretrained vision–language backbone with a flow-based action expert [5], [6]. The OpenPI implementation trains with the linear interpolation

$$x_t = t\epsilon + (1 - t)A, \quad u_t = \epsilon - A, \quad (2)$$

and performs Euler integration from  $t = 1$  to  $t = 0$ ; the released PyTorch sampler uses ten steps by default [2], [7].

Throughout this proposal,  $\theta$  is frozen. We modify only a small steering network with parameters  $\psi$ .

#### B. Executed prefix and controlled first scope

The VLSA runner replans after five executed actions [2]. We therefore begin with the translational prefix

$$A = [a_1, \dots, a_H] \in \mathbb{R}^{H \times 3}, \quad H = 5, \quad (3)$$

while leaving rotation and gripper commands unchanged. The first study uses the same translational-only regime considered in SafeLIBERO experiments [1]. Under the controlled kinematic model,

$$p_{h+1} = p_h + \kappa a_h, \quad h = 0, \dots, H - 1, \quad (4)$$

where  $p_h \in \mathbb{R}^3$  is the end-effector position and  $\kappa$  is fixed by the control rate and action scaling.

#### C. Continuous collision margin

Let  $G$  denote the known obstacle geometry and  $r_{\text{eef}}$  a conservative end-effector radius in the controlled pilot. For an action prefix  $A$ , define the swept signed-distance margin

$$q_{h,\lambda} = (1 - \lambda)p_h + \lambda p_{h+1}, \\ D(A; s, G) = \min_{\substack{h \in \{0, \dots, H-1\} \\ \lambda \in [0, 1]}} [\text{sdist}_G(q_{h,\lambda}) - r_{\text{eef}}]. \quad (5)$$

A prefix is collision-free when  $D(A; s, G) \geq 0$ . The optimization model may use the known differentiable primitive geometry in the controlled study, but all reported safety outcomes are verified with simulator geometry and contact information at physics-substep resolution.

The endpoint map is

$$E(A; s) = p_H. \quad (6)$$

Endpoint preservation is a deliberately limited proxy for local task intent: the corrected prefix should reach the point that the nominal VLA intended to reach, while taking a different local path.

#### D. Research objective

For a nominal sample  $A^- \sim \pi_\theta(\cdot \mid o, \ell, s)$ , we seek a learned intervention that generates  $\tilde{A}$  satisfying

$$D(\tilde{A}; s, G) \geq 0, \quad E(\tilde{A}; s) = E(A^-; s), \quad (7)$$

with small action change and without an online safety optimizer. The key question is not merely whether collision can be predicted, but whether a direction that realizes Eq. (7) is present and learnable in the action-flow representation.

### IV. PROPOSED METHOD: CRFS

#### A. Minimal safe counterfactual

For each nominal prefix  $A^-$ , define the counterfactual action

$$A^+ = \arg \min_{A \in \mathcal{A}} \frac{1}{2} \|A - A^-\|_F^2 \\ \text{s.t. } D(A; s, G) \geq 0, \\ E(A; s) = E(A^-; s). \quad (8)$$

Here  $\mathcal{A}$  is the valid normalized action range. Equation (8) is an offline label generator, not the learned objective and not an inference-time controller. It contains one optimization criterion: minimum action change. Safety and task preservation are hard constraints rather than weighted penalties.

If Eq. (8) is infeasible, the example is marked infeasible rather than softened with extra penalty terms. If  $A^-$  is already safe, then  $A^+ = A^-$ . The supervision vector is

$$\Delta A^* = A^+ - A^-. \quad (9)$$

Thus safe examples have exactly zero target correction.

#### B. Endpoint-preserving correction subspace

Under Eq. (4), endpoint change is linear in the action correction. Let  $\delta a = \text{vec}(\Delta A) \in \mathbb{R}^{3H}$  and

$$B = \kappa [I_3 \ I_3 \ \dots \ I_3] \in \mathbb{R}^{3 \times 3H}. \quad (10)$$

Endpoint preservation requires  $B\delta a = 0$ . A network may output an unconstrained vector  $z$ , which is projected into the endpoint-preserving null space:

$$\widehat{\delta a} = \Pi_{\text{ep}} z, \quad \Pi_{\text{ep}} = I - B^\top (BB^\top)^{-1} B. \quad (11)$$

For the uniform translational model, Eq. (11) is equivalent to subtracting the mean correction across the  $H$  actions. This removes the need for an endpoint penalty in the training loss.

#### C. Intermediate representation and steering model

The base sampler is run normally from  $t = 1$  to a fixed intervention time  $t_s$ . The first experiment uses  $t_s = 0.5$ , corresponding to the midpoint of the ten-step sampler. At that point we retain

$$x_{t_s}, \quad \widehat{A}_{t_s} = x_{t_s} - t_s v_\theta(x_{t_s}, t_s \mid o, \ell, s), \quad (12)$$

where  $\widehat{A}_{t_s}$  is the current predicted-clean action estimate implied by the training bridge in Eq. (2).

The steering network receives

$$\xi_{t_s} = [x_{t_s}, \widehat{A}_{t_s}, \phi(s, G), t_s] \quad (13)$$

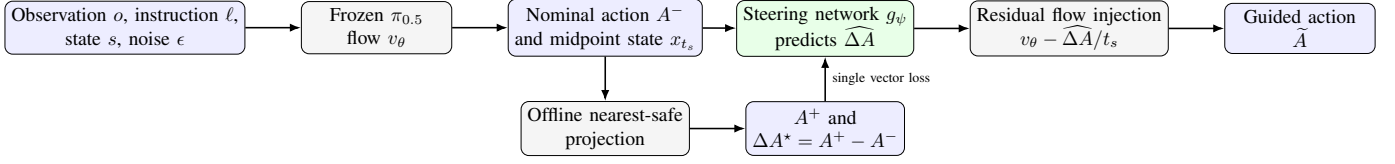


Fig. 1. Overview of CRFS. Constrained optimization is used only offline to construct the minimal safe counterfactual. Deployment predicts a correction from an intermediate flow state and injects it as a residual velocity; no QP or CBF solver is used online.

and predicts a raw correction  $z = g_\psi(\xi_{t_s})$ . The geometry encoding  $\phi(s, G)$  is deliberately simple in the first study: obstacle pose and size in the current end-effector frame. The endpoint-preserving output is

$$\widehat{\Delta A} = \text{unvec}(\Pi_{\text{ep}} z). \quad (14)$$

A small multilayer perceptron is sufficient for the first controlled experiment. Attention maps, image features, and action-expert hidden states are deferred until the basic mechanism is validated.

#### D. Residual flow injection

After predicting  $\widehat{\Delta A}$  once at  $t_s$ , the remaining flow is integrated with

$$\frac{dx_t}{dt} = v_\theta(x_t, t \mid o, \ell, s) - \frac{1}{t_s} \widehat{\Delta A}, \quad t \in [t_s, 0]. \quad (15)$$

The correction is padded with zeros for non-translational dimensions and for actions beyond the executed prefix. Ignoring the change in the nonlinear base velocity caused by the edited state, the cumulative residual is

$$\int_{t_s}^0 -\frac{1}{t_s} \widehat{\Delta A} dt = \widehat{\Delta A}. \quad (16)$$

Hence the residual has a direct interpretation: it attempts to shift the final action by the predicted safe displacement. The equality is only exact for the additive residual term; the frozen base field is reevaluated at the modified states. Whether the nonlinear sampler retains the desired correction is therefore an empirical question tested by the oracle-steering experiment before any learned model is trusted.

With Euler step  $\Delta t < 0$ , the update is

$$x_{k+1} = x_k + \Delta t \left[ v_\theta(x_k, t_k \mid o, \ell, s) - \frac{1}{t_s} \widehat{\Delta A} \right]. \quad (17)$$

#### E. Single training objective

The frozen VLA is not updated. The only learned objective is

$$\mathcal{L}_{\text{CRFS}}(\psi) = \mathbb{E} \left[ \left\| \widehat{\Delta A} - \Delta A^* \right\|_F^2 \right]. \quad (18)$$

No collision classifier, reward scalar, smoothness term, endpoint term, or task-success auxiliary head is used in the first study. Safe nominal examples contribute zero-vector targets.

---

#### Algorithm 1 Offline counterfactual dataset construction

---

**Require:** Frozen VLA  $v_\theta$ , simulator state  $S$ , observation  $o$ , instruction  $\ell$ , obstacle  $G$ , noise  $\epsilon$

- 1: Run the base sampler; save  $x_{t_s}$  and nominal prefix  $A^-$
  - 2: Execute  $A^-$  from  $S$  and measure  $D(A^-; S, G)$
  - 3: **if**  $D(A^-; S, G) \geq 0$  **then**
  - 4:     Set  $A^+ \leftarrow A^-$
  - 5: **else**
  - 6:     Solve Eq. (8) for  $A^+$
  - 7:     **if** infeasible **then**
  - 8:         record infeasible case and stop
  - 9:     **end if**
  - 10:    Verify  $A^+$  in the simulator from the identical state  $S$
  - 11: **end if**
  - 12: Store  $(x_{t_s}, \hat{A}_{t_s}, \phi(S, G), \Delta A^*)$
- 

---

#### Algorithm 2 Learned steering at inference

---

**Require:** Observation  $o$ , instruction  $\ell$ , state  $s$ , obstacle geometry  $G$ , noise  $\epsilon$

- 1: Run frozen sampler from  $t = 1$  to  $t = t_s$
  - 2: Compute  $\hat{A}_{t_s}$  using Eq. (12)
  - 3: Predict  $z \leftarrow g_\psi(x_{t_s}, \hat{A}_{t_s}, \phi(s, G), t_s)$
  - 4: Project  $z$  with Eq. (11) to obtain  $\widehat{\Delta A}$
  - 5: Continue sampling with Eq. (17) until  $t = 0$
  - 6: Return guided action chunk  $\hat{A}$
- 

## V. EXPERIMENTAL VERIFICATION PLAN

The experiments are intentionally staged. A later stage is attempted only if the previous gate passes. This prevents model complexity from hiding a failure in geometry, feasibility, or the intervention point.

#### A. Controlled pilot configuration

The first experiment uses:

- one SafeLIBERO Level-II scenario with an obstacle directly in the movement path;
- one static, asymmetric, convex obstacle with simulator-known geometry;
- translational actions only;
- the first  $H = 5$  executed actions;
- the default ten-step OpenPI flow sampler and  $t_s = 0.5$ ;
- the same simulator state, observation, instruction, and noise for paired comparisons;
- episode-level train/validation/test splits, never random frame splits.

The asymmetry is important. A symmetric obstacle may admit equally short left and right detours, making the correction target multimodal. The first study should not mix a multimodal prediction problem with the more basic question of whether flow steering works at all.

A practical pilot collection target is at least 500 colliding and 500 already-safe nominal prefixes, generated across multiple episode initializations and noise samples. Counts may be increased after measuring the collision prevalence, but all thresholds and splits should be fixed before training the steering model.

### B. Ground-truth safety measurement

The mechanism study uses the minimum signed geometry distance over every physics substep in MuJoCo [8] of the five-action execution. A contact event is cross-checked against simulator contact pairs. This is more informative than the obstacle-displacement flag used in the released benchmark runner, which marks a collision after the obstacle position changes by more than a fixed threshold [2]. The original benchmark metric should still be reported later for comparability, but it should not be the supervision source for the proposed method.

Before collecting data, the following instrumentation tests are mandatory:

- 1) Restore the same simulator state and execute the same action prefix at least five times.
- 2) Confirm that the measured minimum signed distance varies by less than  $10^{-4}$  m.
- 3) Confirm that reported first contact occurs near zero signed distance within the simulator's numerical tolerance.
- 4) Visualize the geometry pair achieving the minimum distance and verify that it belongs to the end effector and designated obstacle.

### C. Stage 0: measurement validity

a) *Question.*: Is the continuous safety measurement reproducible and physically correct?

b) *Pass condition.*: All four instrumentation tests above pass on at least 50 diverse saved states.

c) *Stop/refine condition.*: If reproducibility or contact consistency fails, no learning experiment is run. The state-reset mechanism, geometry IDs, or substep tracker must be corrected first.

### D. Stage 1: counterfactual feasibility

a) *Question.*: Does a local endpoint-preserving safe correction exist within the five-action horizon?

For every colliding nominal prefix, solve Eq. (8). Report

$$F_{\text{proj}} = P(A^+ \text{ is feasible} \mid D(A^-) < 0), \quad (19)$$

$$S_{\text{teacher}} = P(D(A^+) \geq 0 \mid A^+ \text{ feasible}), \quad (20)$$

and the median endpoint error after simulator execution,

$$E_{\text{teacher}} = \text{median} \|E(A^+) - E(A^-)\|_2. \quad (21)$$

b) *Pilot pass condition.*:

$$F_{\text{proj}} \geq 0.60, \quad S_{\text{teacher}} \geq 0.95, \quad E_{\text{teacher}} \leq 5 \text{ mm}. \quad (22)$$

These are pre-registered pilot gates, not universal safety standards.

c) *Stop/refine condition.*: If  $S_{\text{teacher}} < 0.95$ , the label construction is not trustworthy and the solver or geometry model must be fixed. If  $F_{\text{proj}} < 0.40$ , repeat only once with  $H = 10$ . If feasibility remains below 0.60, the local endpoint-preserving premise is considered false for this task; the project should pivot to longer-horizon planning or earlier intervention rather than adding a more complex network.

### E. Stage 2: oracle flow steerability

a) *Question.*: If the exact correction  $\Delta A^*$  is known, can the residual flow in Eq. (15) turn the same observation-noise sample into a safe action?

For each feasible colliding example, compare:

- 1) **Base**: no intervention;
- 2) **Random residual**: an endpoint-preserving random vector with the same norm as  $\Delta A^*$ ;
- 3) **Oracle residual**: the exact  $\Delta A^*$ ;
- 4) **Direct oracle action**: execute  $A^+$  directly, serving only as an upper bound.

All conditions use the same saved state and the same initial flow noise.

Let  $\tilde{A}_{\text{oracle}}$  and  $\tilde{A}_{\text{rand}}$  denote the final sampled actions. The primary quantities are

$$R_{\text{oracle}} = P(D(\tilde{A}_{\text{oracle}}) \geq 0 \mid D(A^-) < 0, A^+ \text{ feasible}), \quad (23)$$

$$R_{\text{rand}} = P(D(\tilde{A}_{\text{rand}}) \geq 0 \mid D(A^-) < 0, A^+ \text{ feasible}), \quad (24)$$

and the median local endpoint error of the oracle-guided action.

b) *Pilot pass condition.*: The oracle intervention is considered meaningfully steerable when

$$R_{\text{oracle}} \geq 0.50, \quad R_{\text{oracle}} - R_{\text{rand}} \geq 0.20, \quad (25)$$

with a paired 95% bootstrap confidence interval for the difference strictly above zero, and with median endpoint error below 1 cm.

c) *Stop/refine condition.*: If the midpoint intervention fails, test only two later intervention times,  $t_s \in \{0.3, 0.1\}$ . If no intervention time significantly exceeds the equal-norm random residual, action-flow steering is rejected for this formulation. The next research direction should change the intervention representation or directly adapt the action expert, rather than train a predictor for a non-causal direction.

### F. Stage 3: learned correction

a) *Question.*: Can the correction be predicted from the intermediate flow state and simple obstacle geometry?

Train  $g_\psi$  with Eq. (18). The test set must contain held-out episode initializations and obstacle poses. The mandatory comparisons are:

- 1) base  $\pi_{0.5}$ ;
- 2) equal-norm random residual;
- 3) best-of-8 noise seeds, to test whether sampling alone can find safe support;
- 4) learned CRFS;
- 5) oracle residual, as the achievable upper bound for this intervention mechanism.

For colliding nominal samples, define the learned rescue rate

$$R_{\text{learned}} = P(D(\tilde{A}_{\text{learned}}) \geq 0 \mid D(A^-) < 0). \quad (26)$$

For already-safe nominal samples, define safe preservation

$$P_{\text{preserve}} = P(D(\tilde{A}_{\text{learned}}) \geq 0 \mid D(A^-) \geq 0). \quad (27)$$

The fraction of the oracle steering gap recovered is

$$\rho = \frac{R_{\text{learned}} - R_{\text{rand}}}{R_{\text{oracle}} - R_{\text{rand}}}. \quad (28)$$

*b) Pilot pass condition.:* Proceed to closed-loop evaluation when

$$\rho \geq 0.50, \quad P_{\text{preserve}} \geq 0.95, \quad (29)$$

and the learned intervention maintains median endpoint error below 1 cm. Learned rescue must also significantly exceed best-of-8 seed selection on the subset where all eight base samples are unsafe.

*c) Stop/refine condition.:* If training error is low but test rescue is poor, the issue is generalization; add obstacle-pose and state diversity before changing the loss. If training error itself is high, inspect target multimodality and input sufficiency. Permit only two refinements before reassessing the premise: doubling the dataset and one architecture/input revision. If  $\rho < 0.30$  after both, the correction is not sufficiently predictable from the chosen intermediate representation and the idea should pivot to a richer representation or direct residual-flow fine-tuning.

#### G. Stage 4: closed-loop utility

*a) Question.:* Does the learned intervention improve complete task execution rather than only five-action clips?

Run full episodes with:

- 1) base  $\pi_{0.5}$ ;
- 2) best-of-8 seed selection;
- 3) learned CRFS;
- 4) AEGIS as an external analytic safety reference;
- 5) optionally, CRFS followed by AEGIS, to test complementarity rather than replacement.

The primary episode metric is

$$\text{SafeSuccess} = P(\text{task succeeds} \wedge \text{no forbidden collision occurs}). \quad (30)$$

Collision avoidance rate and ordinary task success rate are reported separately because a policy can appear safe by stopping, and can complete a task despite an unsafe collision. The VLSA benchmark likewise reports collision avoidance and task success separately [1].

*b) Pilot pass condition.:* The direction is considered worthy of broader study when learned CRFS improves SafeSuccess by at least five percentage points over the base policy with a paired 95% confidence interval above zero, while ordinary task success decreases by no more than five percentage points. The adapter should also add less than 10% to VLA inference latency.

*c) Stop/refine condition.:* If clip-level rescue succeeds but closed-loop SafeSuccess does not improve, inspect whether corrections drive the policy into out-of-distribution states, a failure mode also discussed for safety interventions in VLSA [1]. One data-aggregation round is allowed: collect states reached by the guided policy, rebuild counterfactual labels there, and retrain. If SafeSuccess still does not improve, the method is not worthwhile in its current local form.

## VI. DECISION TABLE

## VII. STATISTICAL PROTOCOL

All mechanism comparisons are paired by simulator state and flow noise. Confidence intervals are obtained by bootstrapping complete episodes or state groups, not individual frames. Seeds from the same state must remain in the same train, validation, or test split. Report absolute rates, paired differences, and 95% confidence intervals. Hyperparameters, intervention times, and pass criteria are fixed before observing test results.

The strongest evidence against the seed-selection explanation is the *all-sampled-unsafe* subset. For each state, draw eight base noise samples. Retain groups for which all eight nominal prefixes collide. A successful learned residual on these groups demonstrates that the method is not merely selecting an already-safe base sample.

## VIII. IMPLEMENTATION PLAN IN THE RELEASED CODEBASE

The implementation requires three localized changes:

- 1) **Expose the intervention state.** Modify the OpenPI PyTorch sampler to return  $x_{t_s}$  and the velocity at the chosen Euler step. The current implementation initializes  $x_t$  with Gaussian noise, uses ten Euler steps by default, and returns only the final action chunk [2], [7].
- 2) **Add residual integration.** After  $t_s$ , subtract  $\widehat{\Delta A}/t_s$  from the translational prefix of the predicted velocity at each remaining Euler step.
- 3) **Branch and measure.** From a saved simulator state, execute nominal, random, oracle, and learned actions separately. Track minimum end-effector–obstacle signed distance at every physics substep. The released runner already exposes a five-step replanning parameter and a safe-success log, but its displacement-based collision flag should not be used as the continuous supervision target [2].

No change to the pretrained VLA weights is required in the first version. The steering network can be trained independently from stored intermediate states and counterfactual vectors.

TABLE II  
PRE-REGISTERED DECISION GATES. THRESHOLDS ARE PILOT CRITERIA CHOSEN TO PREVENT INDEFINITE ITERATION; THEY ARE NOT CLAIMS OF DEPLOYMENT-LEVEL SAFETY.

| Stage | Core question                             | Continue criterion                                                                               | Stop or refine action                                                                                      |
|-------|-------------------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 0     | Is the signed-distance measurement valid? | Repeatability $< 10^{-4}$ m and contact-consistent geometry on 50 states                         | Stop all model work; repair state restoration, geom IDs, or substep measurement                            |
| 1     | Does a local safe equivalent exist?       | $F_{\text{proj}} \geq 0.60$ , teacher safety $\geq 0.95$ , endpoint error $\leq 5$ mm            | If low feasibility, try $H = 10$ once; if still low, pivot to longer-horizon planning                      |
| 2     | Is the flow causally steerable?           | $R_{\text{oracle}} \geq 0.50$ , at least 20-point gain over random, endpoint error $\leq 1$ cm   | Try $t_s = 0.3$ and $0.1$ only; if all fail, reject this action-flow intervention                          |
| 3     | Is the direction learnable?               | $\rho \geq 0.50$ , safe preservation $\geq 0.95$ , and beats best-of-8 on all-unsafe groups      | Double data and change input/architecture once; if $\rho < 0.30$ , pivot representation or training method |
| 4     | Is it useful closed loop?                 | SafeSuccess +5 points or more, confidence interval above zero, task-success loss $\leq 5$ points | One guided-state data-aggregation round; if still no gain, stop the local method                           |

## IX. WHAT IS DELIBERATELY DEFERRED

The following additions are scientifically reasonable but should be attempted only after the five gates in Table II:

- full rotational actions and gripper corrections;
- action-expert hidden states or attention maps as steering inputs;
- multiple obstacles and full-arm collision geometry;
- learned visual obstacle perception instead of simulator-known geometry;
- multimodal corrections around symmetric obstacles;
- repeated feedback prediction of  $\widehat{\Delta A}$  at every flow step;
- real-robot deployment.

This ordering is important. A negative result at an earlier gate provides actionable information. Adding more labels, auxiliary losses, or perception modules before establishing oracle steerability would make the source of failure ambiguous.

## X. LIMITATIONS AND NON-CLAIMS

CRFS has no formal forward-invariance guarantee. The offline projection may be infeasible, the learned correction may generalize poorly, and endpoint preservation is only a local proxy for task intent. The controlled first study covers end-effector translation around one static obstacle and does not address arm-link collisions, dynamic agents, grasp contact semantics, or global planning. The method may also induce distribution shift by moving the robot into states rarely visited by the base policy. These limitations are not hidden by the experimental design; they are exactly what the stage-wise gates are intended to expose.

## XI. EXPECTED RESEARCH OUTCOME

A successful result would support the following limited claim:

For a controlled collision-avoidance setting, an intermediate action-flow state of a frozen  $\pi_{0.5}$  policy admits an endpoint-preserving safety displacement, and a compact learned residual can recover a substantial fraction of the oracle steering effect without online constrained optimization.

A failure is also informative. Low projection feasibility rejects the local-detour assumption; failed oracle steering rejects the chosen flow intervention; failed learning identifies a representation or multimodality bottleneck; failed closed-loop utility exposes safety-induced distribution shift. The proposal is therefore designed to determine quickly whether the idea deserves expansion rather than to accumulate engineering complexity around an unverified premise.

## REFERENCES

- [1] S. Hu, Z. Liu, S. Liu, J. Cen, Z. Meng, S. Wang, X. Li, and X. He, “Vlsa: Vision-language-action models with plug-and-play safety constraint layer,” *arXiv preprint arXiv:2512.11891*, 2025, version 2, July 2026. [Online]. Available: <https://arxiv.org/abs/2512.11891>
- [2] THU-RCSCT, “Vlsa-aegis code and safelibero evaluation,” GitHub repository, 2026, accessed July 2026. [Online]. Available: <https://github.com/THU-RCSCT/vlsa-aegis>
- [3] W. English, H. Zheng, and R. Ewetz, “Neuro-symbolic safety guidance for vision-language-action models via constrained flow matching,” *arXiv preprint arXiv:2607.01378*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.01378>
- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.02747>
- [5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn *et al.*, “ $\pi_0$ : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.24164>
- [6] K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail *et al.*, “ $\pi_{0.5}$ : A vision-language-action model with open-world generalization,” in *Proceedings of the 9th Conference on Robot Learning*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [7] Physical Intelligence, “Openpi: Open-source models and packages for robotics,” GitHub repository, 2026, accessed July 2026. [Online]. Available: <https://github.com/Physical-Intelligence/openpi>
- [8] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.